The Journal of Open Source Software

CurvilinearGrids.jl: A Julia package for curvilinear coordinate transformations

Samuel C. Miller [1]

1 Laboratory for Laser Energetics, University of Rochester, Rochester, NY USA

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Mehmet Hakan Satman

Reviewers:

- @hemanthgrylls
- @DanielVandH

Submitted: 18 November 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Finite-difference discretizations of partial differential equations are widespread throughout the scientific community. Oftentimes finite-differences are used to compute spatial gradients of fields on a discrete grid, which is typically a uniform or rectilinear Cartesian mesh. *Arbitrary* multidimensional geometry is difficult to discretize directly with finite differences, however, due to non-uniform grid spacing and non-orthogonality. Curvilinear coordinate transformations can be used as a strategy to enable arbitrary geometry – see for example, applications to the Euler and Navier-Stokes equations in References (Chandravamsi et al., 2023; Visbal & Gaitonde, 2002). While these curvilinear transformations are straightforward, the governing PDEs require additional terms (metrics) and must adhere to strict conservation laws; these criteria complicate the application of the transformation and require careful implementation.

Statement of Need

`CurvilinearGrids.jl` is a Julia package designed to facilitate curvilinear coordinate transformations and the generation of metrics for arbitrary 1D, 2D, and 3D geometry. The API is designed for simple grid construction with minimal user effort; the backend handles the complicated book-keeping associated with metric computation and ensures strict adherance to geometric conservation laws when required. While Julia packages exist to facilitate standard finite-differences, there are few, if any, that provide conservative metrics for curvilinear transformations.

Users provide `CurvilinearGrids.jl` with discrete coordinate points defined in real-space (x, y, z), or functional forms, e.g `x(i,j)`, `y(i,j)`. These coordinates are then used to compute the transformation (via Jacobian matrices) to a uniform computational coordinate space (ξ, η, ζ). A common example of this is to use a body-fit or conformal grid and transform it so that it becomes a uniform grid in (ξ, η, ζ). Standard finite-difference stencils can be used on the uniform transformed grid. Figure 1 shows an example of a curvilinear grid defined in real-space and it's representation in computational space.

DRAFT

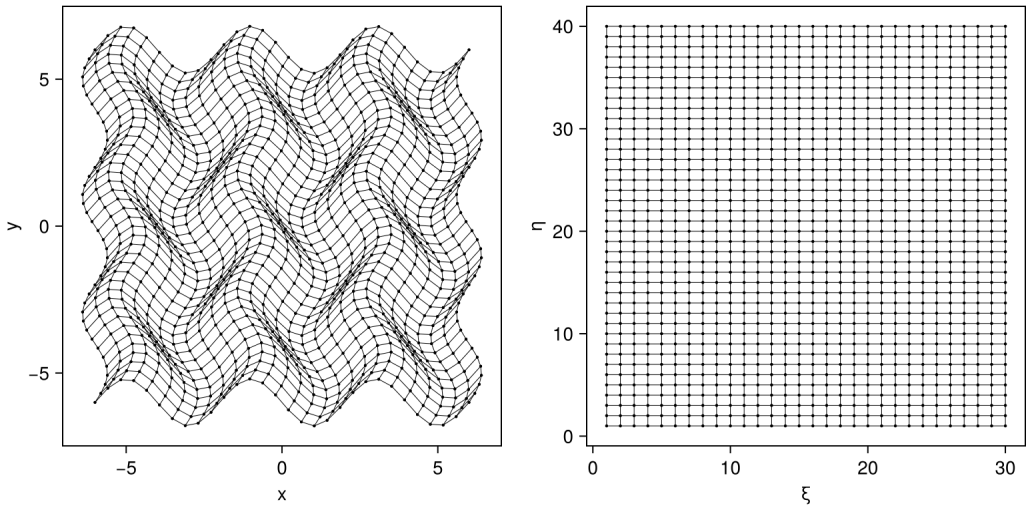

Figure 1: Curvilinear grid transformation.

The grid in Figure 1 is generated by the following code snippet.

```
using CurvilinearGrids

function wavygrid(ni, nj, nhalo=1)
  Lx = Ly = 12
  n_xy = n_yx = 6
  xmin, ymin = (-Lx / 2, -Ly / 2)
  Δx0,  Δy0 = (Lx / (ni - 1), Ly / (nj - 1))
  Ax, Ay = (0.4 / Δx0, 0.8 / Δy0)

  # coordinate arrays
  x = zeros(ni, nj)
  y = zeros(ni, nj)

  @inbounds for idx in CartesianIndices(x)
    i,j = idx.I
    x[i, j] = xmin + Δx0 * (
        (i - 1) + Ax * sinpi((n_xy * (j - 1) * Δy0) / Ly)
    )

    y[i, j] = ymin + Δy0 * (
        (j - 1) + Ay * sinpi((n_yx * (i - 1) * Δx0) / Lx)
    )
  end

  return CurvilinearGrid2D(x, y, nhalo)
end

# number of nodes/vertices in each dimension
nx, ny = (30, 40)

nhalo = 2 # halo cells needed for stencils (can be set to 0)
grid = wavygrid(nx, ny, nhalo)
```

DRAFT

Coordinate Transformations

Coordinate transformations require Jacobian matrices; terminology in the literature can be conflicting, but the "Jacobian matrix" is the matrix of partial derivatives that describe the forward or inverse transformation, and uses a bold-face $\mathbf{J}$. The "Jacobian" then refers to the determinant of the Jacobian matrix, and is the non-bolded J. Some authors refer to the matrix as the "Jacobi matrix" as well. The forward transformation, or $T : (\xi, \eta, \zeta) \to (x, y, z)$ is defined as

$$\mathbf{J} = \begin{bmatrix} x_\xi & x_\eta & x_\zeta \\ y_\xi & y_\eta & y_\zeta \\ z_\xi & z_\eta & z_\zeta \end{bmatrix}, \quad J = \det[\mathbf{J}]$$

The inverse transformation T^{-1}: $(x, y, z) \to (\xi, \eta, \zeta)$ is defined as

$$\mathbf{J}^{-1} = \begin{bmatrix} \xi_x & \xi_y & \xi_z \\ \eta_x & \eta_y & \eta_z \\ \zeta_x & \zeta_y & \zeta_z \end{bmatrix}, \quad J^{-1} = \det[\mathbf{J}^{-1}]$$

Grid Metrics

When solving transformed PDEs in computational coordinates (ξ, η, ζ), grid metrics **must** be included. Care must be taken if advection or motion is included in the governing equations so that the geometric conservation law (GCL) is observed (Thomas & Lombard, 1979). If the scheme does not follow the GCL, errors will build up and ultimately corrupt the solution. In the case of fluid dynamics, the mesh and governing equation discretizations must follow the same scheme and be *conservative* (Visbal & Gaitonde, 2002). One such conservative scheme is the Monotone Explicit Gradient (MEG) based reconstruction (Chamarthi, 2023; Chandravamsi et al., 2023), which is included in `CurvilinearGrids.jl`. Other common schemes include weighted essentially non-oscillatory (WENO) schemes of various order (Ma et al., 2024). There is no restriction to particular schemes which can be added in the future.

The grid metrics (derivative terms in $\mathbf{J}, \mathbf{J}^{-1}$) at each cell-center or edge are accessed through the `AbstractCurvilinearGrid` types exported by `CurvilinearGrids.jl`. The API currently supports 1D, 2D, and 3D geometry, with axisymmetric modes for 1D (spherical and cylindrical) and 2D (cylindrical RZ). Metrics are contained in `StructArrays` for each dimension; **edge metrics** are *conservative* and have the "ˆ" notation, e.g. $\hat{\xi}_x \equiv \xi_x / J$, and **cell-centered metrics** are *non-conservative*, e.g., simply $\xi_x, \eta_x, \ldots$. Chapter 3 in (Huang & Russell, 2011) has a particulariy lucid description of how these metrics can be included in PDE discretizations.

Each grid type includes example metrics like the following subset:

- Cell centered metrics: η_y, y_ξ via `grid.cell_center_metrics`
- Edge centered *conservative* metrics (`grid.edge_center_metrics`): $\hat{\xi}_{x1}, \hat{\xi}_{x2}, \hat{\xi}_{x3}$ at $i + 1/2, j + 1/2, k + 1/2$
- Temporal metrics: ξ_t, η_t, ζ_t

Both edge-centered and cell-centered metrics are required for many PDE discretizations – the interpolation from cell-center to edge must be consistent, e.g. the same interpolation scheme must be shared by the PDE discretization and mesh metric computation. One such scheme is

DRAFT

the 6th-order MEG scheme below:

$$
\begin{aligned}
\phi_{i+1/2} &= \frac{1}{2}\left(\phi^L_{i+1/2} + \phi^R_{i+1/2}\right) \\
&= \frac{1}{2}\left[\left(\phi_i + \frac{1}{2}\phi'_i + \frac{1}{12}\phi''_i\right) + \left(\phi_{i+1} + \frac{1}{2}\phi'_{i+1} + \frac{1}{12}\phi''_{i+1}\right)\right] \\
\phi_{i-1/2} &= \frac{1}{2}\left(\phi^L_{i-1/2} + \phi^R_{i-1/2}\right) \\
&= \frac{1}{2}\left[\left(\phi_{i-1} + \frac{1}{2}\phi'_{i-1} + \frac{1}{12}\phi''_{i-1}\right) + \left(\phi_i + \frac{1}{2}\phi'_i + \frac{1}{12}\phi''_i\right)\right]
\end{aligned}
$$

Where the first derivative is defined as

$$
\left(\frac{\partial \phi}{\partial \xi}\right)_i = \frac{3}{4}(\phi_{i+1} + \phi_{i-1}) + \frac{3}{20}(\phi_{i+2} + \phi_{i-2}) + \frac{1}{60}(\phi_{i+3} + \phi_{i-3}),
$$

and the second derivative is

$$
\left(\frac{\partial^2 \phi}{\partial \xi^2}\right)_i = 2(\phi_{i+1} - 2\phi_i + \phi_{i-1}) + \frac{1}{2}\left[\left(\frac{\partial \phi}{\partial \xi}\right)_{i+1} - \left(\frac{\partial \phi}{\partial \xi}\right)_{i-1}\right].
$$

At boundaries, one-sided or partially one-sided finite difference equations must be used. In future releases, additional discretization schemes will be included.

Funding

This material is based upon work supported by the Department of Energy [National Nuclear Security Administration] University of Rochester "National Inertial Confinement Fusion Program" under Award Number(s) DE-NA0004144.

This report was prepared as an account of work sponsored by an agency of the United States Government. Neither the United States Government nor any agency thereof, nor any of their employees, makes any warranty, express or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States Government or any agency thereof. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States Government or any agency thereof.

References

Chamarthi, A. S. (2023). Gradient based reconstruction: Inviscid and viscous flux discretizations, shock capturing, and its application to single and multicomponent flows. *Computers & Fluids*, *250*, 105706. https://doi.org/10.1016/j.compfluid.2022.105706

Chandravamsi, H., Chamarthi, A. S., Hoffmann, N., & Frankel, S. H. (2023). On the application of gradient based reconstruction for flow simulations on generalized curvilinear and dynamic mesh domains. *Computers & Fluids*, *258*, 105859. https://doi.org/10.1016/j.compfluid.2023.105859

Huang, W., & Russell, R. D. (2011). *Adaptive Moving Mesh Methods* (Vol. 174). Springer. https://doi.org/10.1007/978-1-4419-7916-2

Ma, W., Luo, D., Li, S., Qiu, J., Ni, G., & Chen, Y. (2024). High-order adaptive multi-resolution method on curvilinear grids I: Finite difference framework. *Journal of Computational Physics*, *498*, 112654. https://doi.org/10.1016/j.jcp.2023.112654

Thomas, P. D., & Lombard, C. K. (1979). Geometric Conservation Law and Its Application to Flow Computations on Moving Grids. *AIAA Journal*, *17*(10), 1030–1037. https://doi.org/10.2514/3.61273

Visbal, M. R., & Gaitonde, D. V. (2002). On the Use of Higher-Order Finite-Difference Schemes on Curvilinear and Deforming Meshes. *Journal of Computational Physics*, *181*(1), 155–185. https://doi.org/10.1006/jcph.2002.7117

DRAFT